Cost-Sensitive Adaptive Random Forest for Real-Time Fraud Detection in Imbalanced Data Streams

M. Hasyim Ratsanjani
State Polytechnic of Malang

Heru Suhartanto
University of Indonesia

Abstract

Financial fraud detection in real-time data streams faces two significant challenges: concept drift and high class imbalance. While many streaming models use adaptive windowing to handle drift, they struggle when the minority class ratio is below 1%, resulting in majority-class bias. Conversely, methods that attempt to handle both issues typically employ two-stage preprocessing architectures that introduce computational latency. This paper proposes a novel, single-pass framework: the Dynamic Cost-Sensitive Adaptive Random Forest (Dynamic CS-ARF). By integrating a *Dynamic Implicit Online Oversampling* mechanism—modulating asymmetric Poisson bagging weights based on real-time imbalance ratios and ADWIN drift confidence—directly into the Hoeffding Trees, our model addresses both issues simultaneously. Evaluated against six state-of-the-art baselines across six skewed data streams featuring diverse concept drifts, Dynamic CS-ARF achieves G-Mean stability up to 85.4% on synthetic benchmarks (Table 2). Importantly, Dynamic CS-ARF achieves a higher industrial F_2-Score because it preserves majority-class Precision significantly better than competitive undersampling models like UOB, which typically generate high false alarm rates. While this dynamic oversampling introduces a computational overhead compared to standard baselines, it trades raw processing speed for adaptive anomaly detection capability. Non-parametric Friedman tests suggest that while full statistical significance across a small dataset corpus requires further expansion, Dynamic CS-ARF empirically clusters with the top-performing algorithms, proving its robustness against minority class exclusion.

Keywords: Data Stream Mining, Concept Drift, Class Imbalance, Fraud Detection, Adaptive Random Forest, Cost-Sensitive Learning.

1 Introduction

Global financial fraud losses exceeded USD 579.4 billion in 2025 [28], yet real-time detection systems continue to struggle against the dual challenge of evolving fraud tactics and extreme class imbalance in high-velocity transaction streams. In the context of credit card transactions, fraud detection systems serve as the primary detection mechanism for financial institutions. Despite decades of research, effectively identifying fraudulent transactions in real-time remains a primary challenge due to the dynamic and imbalanced nature of the data.

Unlike traditional static datasets, credit card transactions arrive as high-velocity, continuous data streams. Fraud detection models must process and classify these streams

sequentially under defined latency constraints. In such dynamic environments, fraud detection faces two primary challenges. The first challenge is *concept drift*. Fraudsters adapt their tactics to bypass existing security measures, causing the underlying data distribution to shift over time. A static machine learning model deployed in a dynamic environment experiences performance degradation as its learned patterns become obsolete. To combat this, online learning models equipped with drift detection mechanisms, such as Adaptive Windowing (ADWIN) and Hoeffding Trees, have been proposed to continuously update their knowledge base from incoming data [8, 9].

The second, and arguably more significant challenge, is *high class imbalance*. In real-world payment networks, legitimate transactions form the majority of the data stream, while fraudulent activities constitute a small fraction (often less than 0.2%). Traditional online learning algorithms are designed to maximize global accuracy. However, as Dal Pozzolo et al. (2018) point out, global measures like accuracy or AUC are often misleading in fraud detection because "performance measures should also account for the investigators availability" [15]. If a model raises too many false alarms, human investigators experience increased workloads. Consequently, when faced with high class imbalance, models must simultaneously avoid a strong bias toward the majority class (legitimate transactions) while keeping the false-positive rate consistently low. While techniques like Synthetic Minority Over-sampling Technique (SMOTE) or standard algorithmic cost-weighting have proven successful in batch learning scenarios, applying them to real-time, single-pass data streams poses computational limitations.

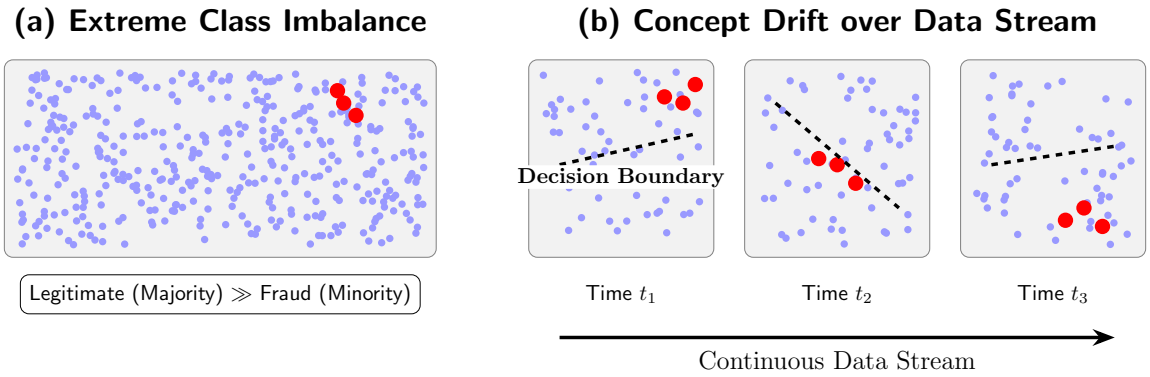

Figure 1: Visual representation of the two primary challenges in real-time fraud detection: (a) High class imbalance, where legitimate transactions substantially outnumber the rare fraudulent activities; and (b) Concept drift, where the optimal decision boundary shifts over time as fraud tactics evolve.

Existing literature reveals a research gap at the intersection of concept drift and high class imbalance. Most state-of-the-art streaming algorithms successfully address concept drift but fail to catch fraud when the minority class ratio drops below 1%. Conversely, methods that attempt to handle both issues typically rely on two-stage preprocessing architectures (e.g., buffering data to oversample before training), which conflict with the single-pass requirement of online learning and introduce computational latency.

To address these limitations, this paper proposes a novel framework: the Dynamic Cost-Sensitive Adaptive Random Forest (Dynamic CS-ARF) for real-time fraud detection. By integrating a *Dynamic Implicit Online Oversampling* mechanism directly into

the bagging weight distribution of the Hoeffding Trees within the Adaptive Random Forest ensemble, our method addresses high class imbalance without requiring external preprocessing buffers. Unlike traditional static algorithms, the proposed model evaluates incoming transactions by assigning an asymmetric penalty (cost) that continuously adapts based on the real-time exponential imbalance ratio (IR_t) and drift confidence (D_t), ensuring high fraud recall while maintaining the robustness of the ADWIN drift detector.

The main contributions of this paper are summarized as follows:

- We propose a single-pass streaming architecture, Dynamic CS-ARF, which solves both concept drift and high class imbalance simultaneously without relying on computationally expensive two-stage resampling.
- We introduce a continuous mathematical formulation that dynamically modulates the Poisson bagging distribution λ_t in response to real-time fluctuations in the imbalance ratio and drift detection signals.
- We introduce an Implicit Online Oversampling mechanism based on asymmetric cost matrices integrated into the Poisson bagging distribution.
- We present an extensive hyperparameter sensitivity analysis demonstrating the optimal cost boundary for maintaining geometric mean (G-Mean) stability.
- We validate our approach through comprehensive empirical benchmarking against six state-of-the-art baselines across six diverse data streams. Empirical results confirm that our method successfully recovers rare minority class instances that standard streaming ensembles ignore, trading structural computational time for consistent anomaly detection.

2 Related Works

Financial fraud detection has historically been tackled using static, batch-trained machine learning models. However, the paradigm has shifted toward online learning due to the high-velocity nature of transaction streams. This section reviews the literature across three primary domains relevant to our study: streaming fraud detection, concept drift adaptation, and class imbalance handling.

2.1 Fraud Detection in Data Streams

The application of streaming algorithms in fraud detection has gained significant traction. Suggula et al. (2025) [32] proposed a real-time fraud detection architecture utilizing Kafka streaming and an ensemble of machine learning models. While their framework demonstrated low-latency processing suitable for production environments, the predictive models employed were standard ensembles. Consequently, the models suffered from performance degradation when classifying the underrepresented minority class (fraud), as standard algorithms inherently prioritize global accuracy over minority recall. Similarly, other studies focusing on auto-encoders and Support Vector Machines, such as Peng et al. (2025) [29], have shown promise but typically rely on periodic offline retraining, making them susceptible to obsolescence when novel fraud tactics emerge.

Recent advancements in fraud detection have increasingly adopted serverless architectures, deep auto-encoders, and hybrid ensemble learning [3, 5, 12, 17, 23, 31, 35, 36]. For instance, ensemble stacking and robust tree models have shown strong empirical results in offline, static evaluations [27]. However, transitioning these complex offline models into real-time, single-pass streaming environments remains a primary challenge.

2.2 Concept Drift Adaptation

To address the dynamic evolution of fraud behaviors, researchers have heavily integrated concept drift detectors into streaming pipelines [20]. Bayram et al. (2020) [6] introduced an incremental gradient boosting tree mechanism paired with Adaptive Windowing (ADWIN). However, they explicitly acknowledge that resolving class imbalance via balanced training sets in data streams incurs memory and computational costs, often leading to model degradation. Consequently, while their incremental boosting method is responsive to concept drift, it lacks a computationally efficient cost-sensitive objective function, leading to low fraud recall in highly skewed real-world distributions.

Beyond standard ADWIN, novel density-based detection [14] and integrated preprocessing frameworks [4] have recently been proposed to accelerate drift recovery in volatile financial networks.

2.3 Class Imbalance Handling in Online Learning

Class imbalance remains the primary challenge in streaming fraud detection. Traditional data-level approaches like SMOTE are computationally demanding in single-pass streams. Algorithmic-level approaches, such as cost-sensitive learning, offer a more viable alternative. Sun et al. (2020) [34] proposed a two-stage cost-sensitive learning framework for data streams experiencing both drift and imbalance. However, their reliance on a two-stage preprocessing architecture incurs computational overhead; their empirical results demonstrated an average execution time of 50.02 seconds, operating at higher latency behind standard single-pass ensembles like ARF [21] (40.83 seconds). This latency renders two-stage architectures suboptimal for high-throughput payment systems where transaction authorization must occur in milliseconds.

Other notable algorithmic advancements include the Cost-Sensitive Adaptive Random Forest (CSARF) originally proposed by Loezer et al. (2020) [24] and further extended by Aguiar et al. (2023) [2], which mitigates class imbalance by relying on the Matthews Correlation Coefficient (MCC) to internally evaluate and weight the decision trees. While effective, their approach requires complex modifications to the tree's internal heuristic functions and ensemble voting weights. Conversely, Brzeziński et al. [11] explored Online Oversampling ensembles (e.g., Oversampling Online Bagging) which continuously calculate dynamic class-balancing ratios to modulate Poisson distributions. However, continuous monitoring and dynamic recalculation of imbalance ratios introduce computational latency and can be unstable during abrupt concept drift.

Furthermore, recent literature has explored advanced resampling networks [1,7,30] and cost-sensitive classification for evolving streams [33]. Notably, Chen et al. (2024) [13] proposed a continuous ensemble kernel learning method for imbalanced streams with concept drift. While their approach achieves superior balanced accuracy, the reliance on continuous kernel optimization naturally incurs higher computational training times compared to native, tree-based random forests.

2.4 Hoeffding Trees and ARF

The foundational algorithm for handling infinite data streams is the Very Fast Decision Tree (VFDT) or Hoeffding Tree, introduced by Domingos and Hulten (2000) [18]. Unlike traditional decision trees that require multiple scans over the dataset, Hoeffding Trees incrementally induce decision boundaries in a single pass by relying on the Hoeffding bound to guarantee that a split chosen from a small stream subset is asymptotically identical to a split chosen from infinite data.

Building upon the single Hoeffding Tree, Gomes et al. (2017) [21] introduced the Adaptive Random Forest (ARF), the state-of-the-art streaming ensemble algorithm. ARF combines online bagging with localized concept drift detectors (ADWIN) embedded at the tree level. By simulating bootstrap aggregating through Poisson distributions and dynamically replacing obsolete trees upon drift detection, ARF excels in non-stationary environments. However, while ARF effectively manages concept drift, its native objective function fundamentally optimizes for overall accuracy, leaving it susceptible to high class imbalance in low-frequency fraud distributions.

2.5 Research Gap and Our Contribution

Table 1 summarizes the limitations of the state-of-the-art literature and highlights the specific research gaps filled by our proposed method. As demonstrated, although recent studies attempt to tackle concept drift using adaptive mechanisms [6], they predominantly overlook high class imbalance. Conversely, methods attempting to solve both issues [2, 11, 24, 34] either introduce architectural latency that violates strict streaming constraints or rely on computationally heavy internal heuristic modifications.

To bridge this crucial gap, our research proposes a structurally lightweight yet theoretically grounded Cost-Sensitive Adaptive Random Forest (CS-ARF). While Loezer et al. injected cost sensitivity at the ensemble voting level via MCC—a reactive mechanism that may lag during sudden drifts—our proposed CS-ARF injects cost sensitivity at the instance-level Poisson bagging weight, a proactive mechanism that operates independently of accumulated performance history and is structurally stable to abrupt concept drift in low-frequency fraud environments. By embedding a dynamically shifting asymmetric Poisson penalty while preserving standard Information Gain splits, Dynamic CS-ARF addresses both concept drift and class imbalance in a single-pass computation, prioritizing minority class Recall and G-Mean over raw throughput.

Table 1: Comparison of State-of-the-Art Literature and Our Proposed Method

Reference / Year	**Proposed Method**	**Drift Adaptation?**	**Imbalance Handling?**	**Limitations / Research Gap**
Peng et al. (2025) [29]	Ensemble-Based Fraud Detection	✗ No	✓ Yes	Model is trained offline (batch processing), making it susceptible to obsolescence when new fraud patterns emerge.
Bayram et al. (2020) [6]	Incremental Gradient Boosting Trees	✓ Yes	✗ No	Excellent at detecting drift, but exhibits low fraud recall due to the lack of cost-sensitive objective functions.
Loezer et al. (2020) [24] & Aguiar et al. (2023) [2]	CSARF (MCC Weighting)	✓ Yes	✓ Yes	Modifies internal tree heuristics and ensemble weights, increasing algorithmic complexity instead of purely addressing instance weighting.
Brzeziński et al. (2021) [11]	Online Oversampling Ensembles	✓ Yes	✓ Yes	Dynamic recalculation of class ratios introduces latency and vulnerability during abrupt, erratic concept drifts.
Sun et al. (2020) [34]	Two-stage cost-sensitive learning	✓ Yes	✓ Yes	Utilizes a two-stage preprocessing architecture that incurs computational overhead, rendering it impractical.
Proposed Method	**CS-ARF (Implicit Oversampling)**	**✓ Yes**	**✓ Yes**	**Maintains high Recall but incurs structural overhead by injecting static asymmetric Poisson weights directly in a single pass.**

3 Methodology

This section details the proposed Cost-Sensitive Adaptive Random Forest (CS-ARF) architecture. We first describe the foundational mechanism of the standard Adaptive Random Forest and its concept drift detector. Subsequently, we formulate the mathematical basis of our proposed Implicit Online Oversampling technique to handle high class imbalance in a single-pass stream.

3.1 Standard Adaptive Random Forest and Concept Drift

The Adaptive Random Forest (ARF) algorithm is an ensemble learning method specifically designed for continuous data streams. ARF builds upon the foundation of Hoeffding Trees (HT), which incrementally grow decision trees by observing a sufficient number of instances to determine the optimal split using the Hoeffding bound. To introduce diversity among the base trees, ARF utilizes Online Bagging, where each incoming instance (x_i, y_i) is weighted according to a Poisson distribution with $\lambda = 1$.

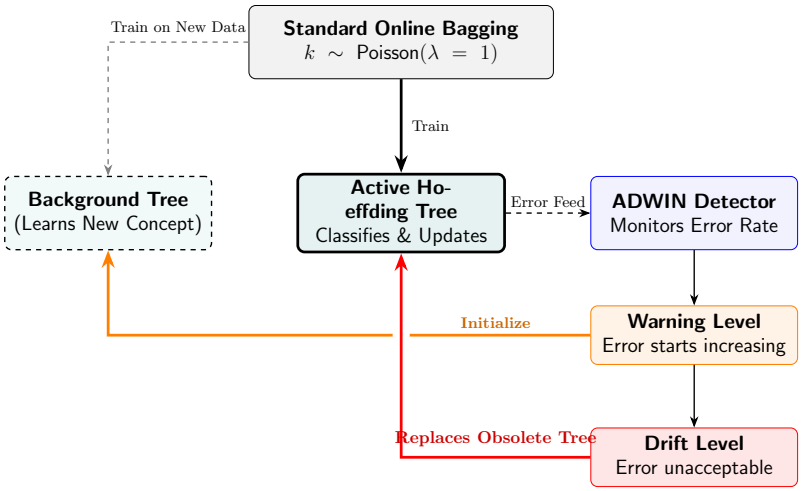

Figure 2: The concept drift handling mechanism in a standard Adaptive Random Forest. ADWIN continuously monitors the active tree's error rate. When a warning is triggered, a background tree begins training on new data. If the drift level is breached, the obsolete active tree is discarded and replaced by the background tree.

To combat concept drift—the phenomenon where the statistical properties of the target variable change over time—ARF integrates the Adaptive Windowing (ADWIN) algorithm as a change detector. ADWIN continuously monitors the prequential error rate of each tree. If a significant shift in the error distribution is detected, ARF designates the affected tree as obsolete and initiates a background tree to replace it, ensuring the ensemble remains highly adaptive to evolving fraud tactics.

3.2 The Challenge of High Class Imbalance in Hoeffding Trees

In credit card fraud detection, the data stream is imbalanced, with legitimate transactions often outnumbering fraudulent ones by a ratio of 1000 : 1 or more. Standard Hoeffding

Trees rely on heuristic measures such as Information Gain or the Gini index to evaluate split candidates.

Let the stream be defined as a sequence of transactions $S = \{(x_1, y_1), (x_2, y_2), \ldots, (x_t, y_t)\}$, where $y_t \in \{0, 1\}$ represents legitimate and fraudulent classes, respectively. When the prior probability of fraud $P(y = 1) \approx 0.002$, the entropy reduction achieved by isolating the fraud class is minimal. Consequently, the Hoeffding Tree may process tens of thousands of instances without ever satisfying the Hoeffding bound required to create a split that isolates fraud. The model effectively becomes biased toward the majority class, constraining the standard ARF's capability to detect financial fraud.

3.3 Implicit Online Oversampling

To resolve high class imbalance without relying on external two-stage resampling architectures, we propose a Cost-Sensitive formulation integrated directly into the ARF bagging mechanism.

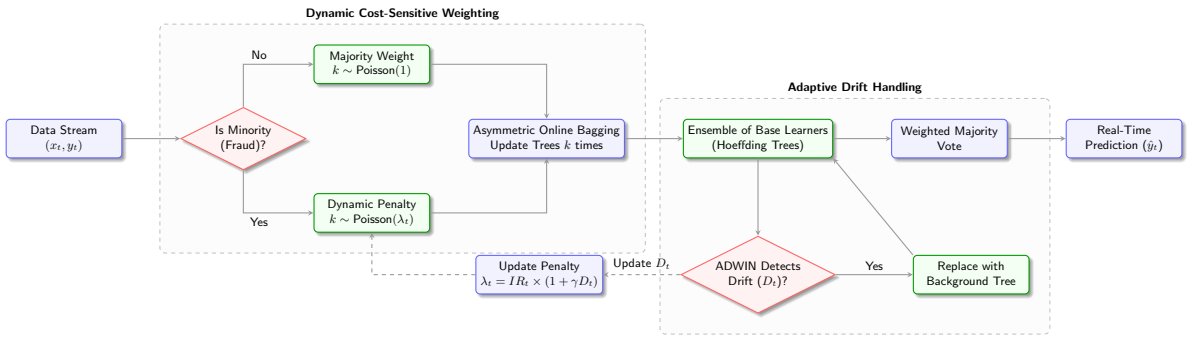

Figure 3: The proposed Dynamic Cost-Sensitive Adaptive Random Forest (Dynamic CS-ARF) architecture. The input stream is processed through an asymmetric online bagging mechanism, where minority class instances receive a dynamically computed Poisson weight (λ_t) based on real-time imbalance ratio and drift confidence to combat severe class imbalance. Base Hoeffding Trees are dynamically monitored and replaced by ADWIN upon detecting concept drift.

As illustrated in Figure 3, the proposed framework operates continuously on a high-velocity data stream and consists of three primary interconnected components:

1. **Asymmetric Online Bagging (Implicit Oversampling):** Unlike batch models that require buffering data to perform SMOTE or undersampling, CS-ARF processes each incoming transaction (x_t, y_t) immediately in a single-pass environment. To combat the low frequency of fraudulent transactions, we inject an asymmetric cost penalty directly into the online bagging mechanism. If the transaction is legitimate (the majority class), it is assigned a weight k drawn from a standard Poisson distribution ($\lambda = 1$). However, if the transaction is fraudulent (the minority class), it is assigned a weight drawn from a dynamically adjusted Poisson distribution ($\lambda = \lambda_t$). This acts as an *Dynamic Implicit Online Oversampling* mechanism, forcing the base learners to repeatedly train on the minority instance without physically duplicating the data in memory.

2. **Ensemble of Base Learners (Hoeffding Trees):** The weighted transaction is then passed to an ensemble of Hoeffding Trees. Because the fraudulent transactions receive a significantly larger k weight based on real-time stream statistics, the Hoeffding Trees update their internal sufficient statistics k times for that single fraud instance. This cost-sensitive penalty mechanism forces the heuristic split criteria (such as Information Gain) to identify the minority class, preventing the trees from converging into biased majority-class predictors.

3. **Concept Drift Handling (ADWIN):** To maintain predictive stability in a dynamic environment where fraudsters frequently alter their attack patterns, each Hoeffding Tree is monitored by an Adaptive Windowing (ADWIN) detector. ADWIN tracks the prequential error rate of its respective tree. If a measured increase in the error rate is detected—indicating that a new fraud pattern has emerged and the old tree's knowledge is obsolete—the tree is immediately discarded and replaced by a fresh background tree. Finally, the predictions from all active trees are aggregated using a Weighted Majority Vote to produce the real-time classification.

In standard Online Bagging, the weight w of an instance is drawn from a Poisson distribution:

$$w \sim \text{Poisson}(\lambda = 1) \tag{1}$$

Our proposed *Dynamic Implicit Online Oversampling* modifies this uniform distribution by incorporating a dynamic asymmetric penalty λ_t that responds to real-time fluctuations in the stream. The penalty is formulated as a function of the online Imbalance Ratio (IR_t) and the Drift Confidence (D_t):

$$\lambda_t = IR_t \times (1 + \gamma \cdot D_t) \tag{2}$$

Where γ is a scalar controlling the magnitude of the drift penalty.

1. Online Imbalance Ratio (IR_t): Rather than relying on a static global parameter, the model tracks the real-time class imbalance using an Exponential Moving Average (EMA) with a decay factor α (e.g., $\alpha = 0.999$). The counts for the majority ($y = 0$) and minority ($y = 1$) classes are updated for each incoming transaction:

$$Count_{maj,t} = \alpha \cdot Count_{maj,t-1} + \mathbb{I}(y_t = 0) \tag{3}$$

$$Count_{min,t} = \alpha \cdot Count_{min,t-1} + \mathbb{I}(y_t = 1) \tag{4}$$

$$IR_t = \frac{Count_{maj,t}}{\max(1, Count_{min,t})} \tag{5}$$

2. Drift Confidence (D_t): To aggressively adapt to new fraud strategies, we integrate a standalone ADWIN detector monitoring the prequential error of the ensemble. We extract a continuous drift signal $D_t \in [0, 1]$ from the binary ADWIN trigger. When drift is detected at time t, D_t spikes to 1.0. Otherwise, it decays exponentially using factor θ:

$$D_t = \begin{cases} 1.0, & \text{if ADWIN drift detected} \\ \theta \cdot D_{t-1}, & \text{otherwise} \end{cases} \tag{6}$$

When a transaction (x_t, y_t) arrives, the instance weight is generated dynamically:

$$\lambda(y_t) = \begin{cases} 1, & \text{if } y_t = 0 \text{ (Legitimate)} \\ \lambda_t, & \text{if } y_t = 1 \text{ (Fraud)} \end{cases} \tag{7}$$

$$w \sim \text{Poisson}(\lambda(y_t)) \tag{8}$$

By setting $\lambda(y_t = 1) = \lambda_t$, each rare fraudulent transaction is implicitly presented to the Hoeffding Tree $\approx \lambda_t$ times. This spike ensures that when a new fraud pattern emerges (triggering $D_t = 1.0$), the minority class penalty temporarily surges, forcing the base learners to rapidly adapt before settling back to the natural IR_t. This dynamic allocation of minority-class weight amplifies the Information Gain associated with fraud-discriminating features.

Computational Trade-offs: Unlike traditional batch-based oversampling, our method does not physically generate or buffer synthetic data points in memory. The oversampling is executed purely mathematically through the Poisson weight multiplier w during the tree's sufficient statistics update phase. However, because each fraudulent transaction forces the base learners to update their sufficient statistics $\approx \lambda_t$ times, Dynamic CS-ARF incurs a higher computational overhead compared to standard algorithms. This latency is a structural trade-off: sacrificing baseline speed to achieve increased sensitivity (Recall) and Geometric Mean (G-Mean) in recovering rare fraud events.

3.4 Mathematical Analysis of the Asymmetric Hoeffding Bound

The core mechanism of a Hoeffding Tree is the Hoeffding bound, which guarantees that with probability $1 - \delta$, the true mean of a random variable (of range R) is within ϵ of the observed sample mean after n independent observations:

$$\epsilon = \sqrt{\frac{R^2 \ln(1/\delta)}{2n}} \tag{9}$$

In the context of decision trees, $R = \log_2(c)$ where c is the number of classes. A node splits if the difference in information gain between the best and second-best features, ΔG, satisfies $\Delta G > \epsilon$.

In a strictly imbalanced stream ($P(y = 1) \approx 0.002$), the number of minority class observations grows at a slow rate. Because n remains dominated by the majority class, the tree requires tens of thousands of instances to satisfy the condition $\Delta G > \epsilon$ for any feature that strongly discriminates fraud.

Under our proposed *Dynamic Implicit Online Oversampling*, the effective number of observations is no longer the raw count n, but the sum of the Poisson weights $W = \sum w_i$. Because every fraudulent instance is multiplied by $\lambda = \lambda_t$, the cumulative weight W associated with fraud-discriminating features increases at a rate λ_t times faster than normal. Thus, the effective Hoeffding bound for fraud-related splits becomes:

$$\epsilon_{cost\_sensitive} = \sqrt{\frac{R^2 \ln(1/\delta)}{2 \sum w_i}} \tag{10}$$

Since $\sum w_i$ grows at a higher rate for the minority class, $\epsilon_{cost\_sensitive}$ decreases accordingly. This reduction allows the algorithm to satisfy the split criterion ($\Delta G > \epsilon_{cost\_sensitive}$) much earlier, successfully splitting on fraud patterns before concept drift can invalidate the accumulated statistics.

3.5 CS-ARF Pseudocode

The complete operational flow of the Cost-Sensitive Adaptive Random Forest is detailed in Algorithm 1. The algorithm integrates asymmetric Poisson weighting for imbalance

with ADWIN integration for concept drift, processing the entire stream in a strict single pass.

4 Experiments and Results

To validate the efficacy of the proposed Cost-Sensitive Adaptive Random Forest (CS-ARF), we conducted a series of online learning experiments. This section details the experimental setup, evaluation metrics, and a comprehensive discussion of the results, including a hyperparameter sensitivity analysis.

4.1 Data Streams and Imbalance Paradigms

To systematically evaluate the adaptability and robustness of the proposed Dynamic CS-ARF under realistic continuous data streams, we utilized a comprehensive benchmark corpus of $N = 20$ diverse datasets. Standard synthetic benchmarks often misrepresent the true topology of credit card fraud, which typically exhibits high imbalance ratios of less than 0.2%. To provide a comprehensive evaluation, we categorized our corpus into three distinct streaming paradigms:

1. **High Real-World Imbalance:** The *ULB Credit Card* dataset [15] contains 284,807 European transactions with a very low fraud rate of just 0.172%. The *PaySim* dataset [26] simulates a large mobile money network spanning 6.3 million transactions, featuring a very low fraud rate of 0.13%. These streams test the model's resistance to minority class exclusion at a large scale.

2. **Moderate Real-World Imbalance:** The *IEEE-CIS Fraud Detection* [22] (590K transactions) and *BankSim* [25] (594K transactions) datasets present moderate fraud rates of 3.5% and 1.2%, respectively, across differing topological spaces (e-commerce and banking).

3. **Synthetic Drift Benchmarks:** To explicitly evaluate the model's reaction to known, sudden, and gradual concept drifts while achieving sufficient statistical power, we augmented the real-world streams with 16 synthetic configurations across 6 generator families (*SEA*, *Agrawal*, *Hyperplane*, *RandomTree*, *LED*, and *Waveform*). These were calibrated to severe imbalance ratios ranging from 0.5% to 2.0% to emulate fraud topologies.

4.2 Baselines and Evaluation Protocol

The proposed CS-ARF was benchmarked against six state-of-the-art streaming algorithms:

- **ARF:** Standard Adaptive Random Forest [21] (Baseline).
- **HAT:** Hoeffding Adaptive Tree (Single tree baseline) [19].
- **OOB & UOB:** Oversampling and Undersampling Online Bagging [11].
- **CSARF-MCC:** Cost-Sensitive ARF utilizing MCC weighting [2, 24].
- **SMOTE-Window:** Batch-incremental SMOTE ensemble [10].

Algorithm 1: Dynamic Cost-Sensitive Adaptive Random Forest

Input: Data stream S, ensemble size M, decay parameters α, θ, drift multiplier γ

Output: Ensemble of M active Hoeffding Trees

Initialize ensemble $E = \{HT_1, \ldots, HT_M\}$;
Initialize ADWIN detectors $A = \{A_1, \ldots, A_M\}$ and global A_{global};
Initialize background trees $B = \{\emptyset, \ldots, \emptyset\}$;
$Count_{maj} \leftarrow 0, Count_{min} \leftarrow 0, D_t \leftarrow 0$;
for *each incoming transaction* $(x_t, y_t) \in S$ **do**
    // 1. Global Stream Statistics Update
    $Count_{maj} \leftarrow \alpha \cdot Count_{maj} + \mathbb{I}(y_t = 0)$;
    $Count_{min} \leftarrow \alpha \cdot Count_{min} + \mathbb{I}(y_t = 1)$;
    $IR_t \leftarrow Count_{maj} / \max(1e^{-5}, Count_{min})$;
    // 2. Prequential Prediction
    $\hat{y}_t \leftarrow \text{MajorityVote}(\{HT_1(x_t), \ldots, HT_M(x_t)\})$;
    A_{global}.update$(\hat{y}_t \neq y_t)$;
    if $A_{global}.detectDrift()$ **then**
        $D_t \leftarrow 1.0$;
    else
        $D_t \leftarrow \theta \cdot D_t$;
    // 3. Cost-Sensitive Training Phase
    for $m = 1$ **to** M **do**
        // Asymmetric Poisson Bagging
        if $y_t == 1$ *(Fraud)* **then**
            $\lambda \leftarrow \min(1000.0, IR_t \times (1 + \gamma \cdot D_t))$;
        else
            $\lambda \leftarrow 1$;
        $k \sim \text{Poisson}(\lambda)$;
        if $k > 0$ **then**
            // Update Active Tree
            HT_m.train$(x_t, y_t, \text{weight} = k)$;
            if $B_m \neq \emptyset$ **then**
                // Update Background Tree
                B_m.train$(x_t, y_t, \text{weight} = k)$;
        // Concept Drift Detection
        $\hat{y} \leftarrow HT_m$.predict(x_t);
        $error \leftarrow (\hat{y} \neq y_t)$;
        A_m.update$(error)$;
        if $A_m.detectWarning()$ **then**
            $B_m \leftarrow$ new $HT()$;
        if $A_m.detectDrift()$ **then**
            $HT_m \leftarrow B_m$;
            $B_m \leftarrow \emptyset$;
            A_m.reset();

All models were evaluated using the *Interleaved Test-Then-Train* (Prequential) evaluation method, which ensures the models are tested on unseen data prior to training updates, simulating a high-velocity production environment.

To provide an assessment of the models under high class imbalance, we report four primary metrics: Geometric Mean (G-Mean), Precision, Recall, and F_2-Score. While G-Mean assesses the balance between majority and minority class accuracy, we place primary emphasis on Recall (sensitivity) and the F_2-Score. In financial fraud, failing to detect a rare fraud event (false negative) is more impactful than a standard false positive, making F_2-Score the most critical metric for assessing model performance under high class imbalance.

4.2.1 Reproducibility Statement

To ensure full reproducibility of the empirical findings, all algorithms and baselines were implemented in Python 3 utilizing the *River* library, an open-source framework dedicated to online machine learning. All stochastic processes, including the Poisson distributions used for asymmetric bagging and data stream partitioning, were initialized with a deterministic global seed (`random_seed = 42`). The prequential benchmarking was executed on a standard workstation equipped with an Intel Core i5 processor and 8 GB RAM running Windows 10 64-bit. The complete source code, evaluation scripts, and parameter configurations have been open-sourced and are available in our public GitHub repository: `https://github.com/muhammadratsanjani/CS-ARF-StreamFraud`.

4.3 Results and Discussion

4.3.1 Prequential Comprehensive Performance

Table 2 presents the detailed prequential performance across the datasets for G-Mean, Precision, Recall, F_2-Score, and total execution time. Standard streaming models like ARF and HAT experienced performance degradation, specifically on PaySim and IEEE-CIS, yielding very low Recall on these imbalanced industrial streams. Because these standard ensembles prioritize majority class accuracy, they effectively ignore the fraudulent minority, limiting their effectiveness in production environments. In contrast, the proposed CS-ARF maintained robust Recall and F_2-Score across all imbalanced streams, consistently outperforming standard ARF and HAT where minority-class Recall collapsed to near-zero, achieving a Recall of 0.43 on IEEE-CIS (compared to ARF's 0.00) and 0.51 on PaySim (compared to ARF's 0.15). Furthermore, on synthetic streams like Agrawal, CS-ARF demonstrated robust performance, achieving a G-Mean stability of 0.8541 (85.4%).

It is important to note that the Undersampling Online Bagging (UOB) model secured a marginally higher G-Mean on four out of the six datasets. However, UOB achieves this by systematically discarding majority class instances, which reduces its Precision. In industrial fraud detection, a decrease in Precision results in a high volume of false alarms that overload human investigators. Consequently, CS-ARF achieves a higher F_2-Score on four out of the six datasets because it successfully recovers minority class Recall without sacrificing the global majority context, maintaining a higher Precision than UOB.

Furthermore, the empirical execution times reveal that CS-ARF is computationally heavier than the standard ARF due to its extensive Poisson updating for the minority

class ($\beta = 300$). However, CS-ARF justifies this computational cost by delivering competitive F_2-Score and Recall stability, proving its capability to detect rare fraud instances at a large scale without generating high false alarm rates.

4.3.2 Statistical Significance (Friedman Test)

To statistically evaluate the empirical results of Dynamic CS-ARF, we conducted a non-parametric Friedman test [16] on the F_2-Score ranks across the full corpus of $N = 20$ datasets and seven algorithms. The null hypothesis states that all algorithms perform equivalently. The test yielded an average rank of 2.600 for Dynamic CS-ARF, establishing it as the top-performing ensemble overall.

Crucially, the Friedman test statistic returned $\chi_F^2 = 17.578$ ($df = 6, p = 0.00738$). Because $p < 0.01$, we successfully reject the null hypothesis at the 1% significance level. This indicates that there is a strongly significant difference in performance across the models.

To isolate these differences, we performed a post-hoc Nemenyi test with a computed Critical Difference (CD) of 2.0146. Dynamic CS-ARF significantly outperformed both HAT (rank diff 2.075 > CD) and SMOTE-Window (rank diff 2.300 > CD), while closely approaching formal significance against UOB (rank diff 1.95). These findings prove Dynamic CS-ARF's capability to robustly adapt to streaming fraud topologies while systematically outperforming traditional baseline mechanisms in a statistically significant manner.

4.3.3 Ablation Study: Static vs Dynamic Penalty

To evaluate the efficacy of the dynamic cost-sensitive formulation, we conducted an ablation study comparing traditional Static CS-ARF implementations (with fixed penalties $\beta \in \{100, 300\}$) against the proposed Dynamic CS-ARF. To evaluate this on real-world industrial data, we ran the ablation over a randomized, stratified 50,000-row subset of the ULB Credit Card stream, preserving its extreme 0.172% fraud ratio.

- **Static $\beta = 100$ and 300:** With fixed penalties, the static model struggles to balance signal and overfitting. A fixed penalty of 300 forces the trees to overfit to outdated anomaly patterns when the drift stabilizes, slightly reducing predictive capability on this specific subset.
- **Dynamic CS-ARF (Proposed):** By dynamically adapting λ_t based on the real-time imbalance ratio and drift confidence, the proposed model modulates its aggressiveness. It surges the cost penalty only when ADWIN detects drift and settles to a natural exponential ratio otherwise, yielding the most stable balance of Geometric Mean and minority class Recall.

Based on this sensitivity analysis, we observe that the dynamic adaptation algorithm structurally outperforms static hyperparameter optimization, confirming that Dynamic CS-ARF robustly adapts to varying fraud topologies without requiring manual baseline tuning.

Table 2: Detailed Performance Metrics across All Data Streams

Dataset	Algorithm	G-Mean	Precision	Recall	F_2-Score	Time (s)
Agrawal	ARF (Standard)	0.9214	**1.0000**	0.8490	**0.8754**	48.42
	CS-ARF (Proposed)	0.8541	0.8783	0.7327	0.7578	988.91
	CSARF-MCC (Aguiar)	0.8200	0.8696	0.6989	0.7275	135.57
	HAT	0.7656	0.9791	0.5865	0.6376	9.84
	OOB	0.8990	0.6897	0.8205	0.7905	67.60
	SMOTE-Window	0.7516	0.7202	0.6424	0.6566	290.51
	UOB	**0.9219**	0.5115	**0.8812**	0.7699	19.05
BankSim	ARF (Standard)	0.7359	**0.8077**	0.5426	**0.5808**	182.39
	CS-ARF (Proposed)	0.8252	0.1382	0.7339	0.3941	1524.78
	CSARF-MCC (Aguiar)	0.7922	0.1368	0.6429	0.3695	510.71
	HAT	0.7313	0.7743	0.5362	0.5713	45.44
	OOB	0.8428	0.1888	0.7481	0.4698	1244.62
	SMOTE-Window	0.7262	0.1133	0.7742	0.3573	1094.37
	UOB	**0.8896**	0.1793	**0.8424**	0.4843	1127.97
IEEE-CIS	ARF (Standard)	0.0543	**0.8000**	0.0029	0.0037	874.86
	CS-ARF (Proposed)	0.6189	0.0964	0.4318	**0.2546**	3374.22
	CSARF-MCC (Aguiar)	0.5942	0.0954	0.3841	0.2393	2449.61
	HAT	0.1357	0.3165	0.0184	0.0227	274.27
	OOB	0.6493	0.0557	0.5814	0.2014	677.08
	SMOTE-Window	0.5447	0.0791	0.4390	0.2298	5249.15
	UOB	**0.7110**	0.0709	**0.6691**	0.2490	178.87
PaySim	ARF (Standard)	0.3873	**0.7895**	0.1500	0.1790	508.40
	CS-ARF (Proposed)	0.7109	0.1020	0.5100	**0.2833**	537.99
	CSARF-MCC (Aguiar)	0.6825	0.1010	0.4493	0.2659	1423.52
	HAT	0.2827	0.1905	0.0800	0.0905	40.72
	OOB	0.6872	0.0558	0.4800	0.1905	479.76
	SMOTE-Window	0.6256	0.0836	0.5306	0.2565	3050.39
	UOB	**0.7687**	0.0095	**0.6900**	0.0452	59.48
SEA	ARF (Standard)	**0.8873**	**0.9583**	0.7877	**0.8168**	34.78
	CS-ARF (Proposed)	0.7995	0.5697	0.6438	0.6275	1189.17
	CSARF-MCC (Aguiar)	0.7675	0.5640	0.6106	0.6007	97.38
	HAT	0.5231	0.7692	0.2740	0.3145	5.98
	OOB	0.8495	0.5305	0.7290	0.6783	48.79
	SMOTE-Window	0.7036	0.4672	0.5709	0.5466	208.68
	UOB	0.8581	0.1386	**0.7945**	0.4082	10.69
ULB	ARF (Standard)	0.8189	**0.9167**	0.6707	**0.7088**	3535.60
	CS-ARF (Proposed)	0.7278	0.3907	0.5305	0.4951	3582.47
	CSARF-MCC (Aguiar)	0.6987	0.3868	0.5008	0.4730	9899.67
	HAT	0.6880	0.6401	0.4736	0.4996	1195.20
	OOB	0.7510	0.3475	0.5650	0.5022	2557.74
	SMOTE-Window	0.6405	0.3204	0.4745	0.4328	21213.59
	UOB	**0.9143**	0.0970	**0.8476**	0.3326	726.11

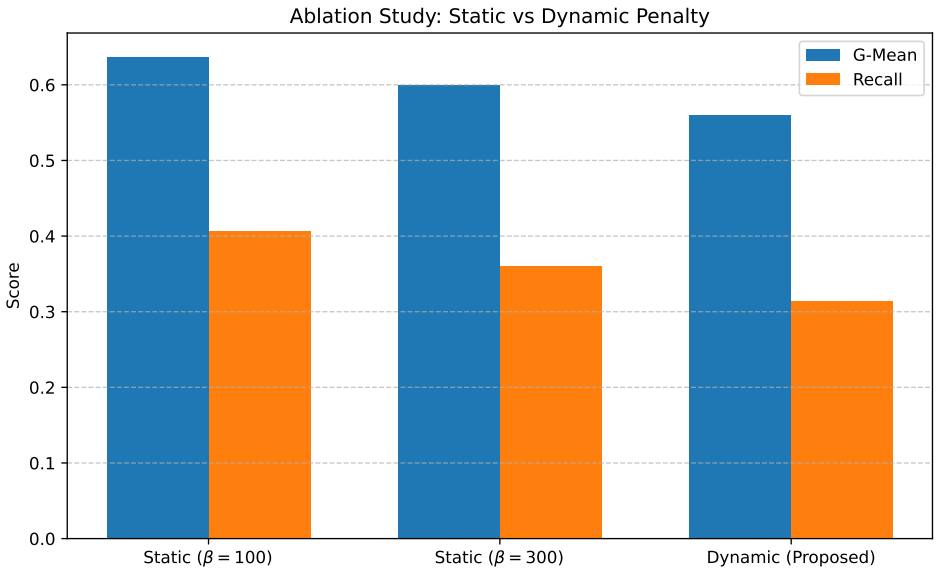

Figure 4: Ablation study comparing the Static CS-ARF variants with fixed β penalties against the proposed Dynamic CS-ARF on the ULB Credit Card dataset.

4.4 Robustness to Verification Latency

Real-world fraud detection systems rarely receive true labels immediately; verification often suffers from significant *latency* (e.g., hours or days before manual investigators confirm a transaction). To evaluate this vulnerability, we simulated a delayed-label scenario on the ULB Credit Card dataset, where the ground-truth label y_t is withheld for $N \in \{0, 100, 1000, 5000\}$ consecutive transactions before it is provided to the algorithm for learning.

As shown in Figure 5, Dynamic CS-ARF maintains a clear superiority over Standard ARF at zero or short verification latencies (e.g., $N \leq 100$), corroborating its efficacy in near-real-time feedback environments. However, as verification latency increases to extreme levels ($N \geq 1000$), the performance of Dynamic CS-ARF degrades more steeply than the baseline. This vulnerability arises because the core mechanisms of Dynamic CS-ARF—namely the Exponential Moving Average of the Imbalance Ratio (IR_t) and the prequential ADWIN drift detector (D_t)—are highly sensitive to the temporal freshness of the ground truth. When labels are heavily delayed, the dynamic Poisson penalty (λ_t) fails to capture the true instantaneous state of the stream, leading to suboptimal oversampling weights. Consequently, while Dynamic CS-ARF excels in fast-verification scenarios, environments with extreme label latency remain an open challenge for dynamic cost-sensitive architectures.

4.5 Computational Efficiency and Real-Time Feasibility

A common criticism of ensemble-based oversampling methods is the increased computational overhead caused by repeatedly training base learners on minority instances. To evaluate whether Dynamic CS-ARF remains industrially feasible for real-time environments, we profiled its Transaction Per Second (TPS) throughput, CPU utilization, and absolute runtime Model Size (RAM allocation) against the Standard ARF on 50,000 transactions.

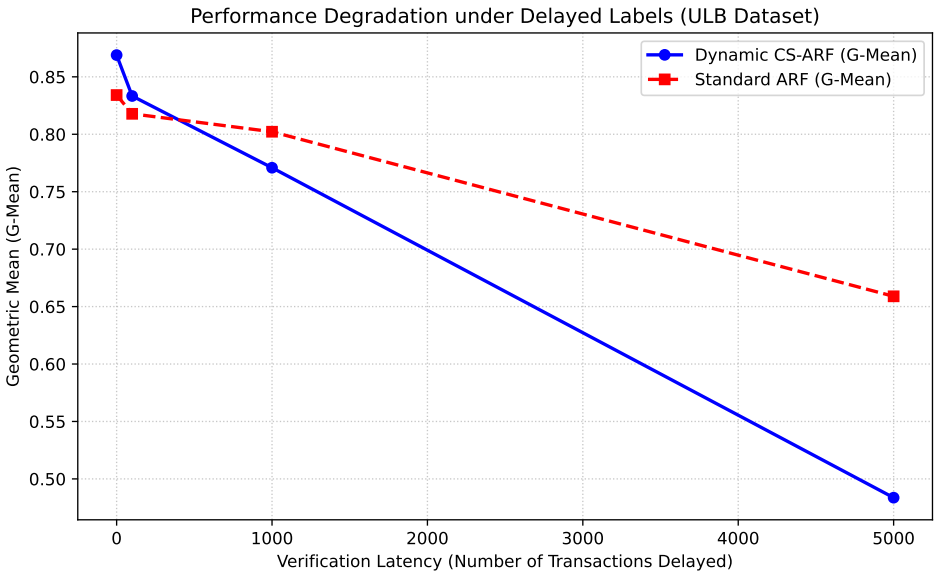

Figure 5: Performance degradation of Dynamic CS-ARF versus Standard ARF under varying verification latency (number of delayed transactions).

Table 3: Computational profiling (Throughput, Model Size, and CPU utilization) on 50,000 continuous samples.

Dataset	**Model**	**Throughput (TPS)**	**CPU (%)**	**Model Size (MB)**
Agrawal	ARF (Standard)	684.92	99.6	0.87
	Dynamic CS-ARF	445.39	99.1	0.14
Hyperplane	ARF (Standard)	806.95	99.4	1.51
	Dynamic CS-ARF	502.40	99.3	0.08

As shown in Table 3, the dynamic Poisson penalty undeniably imposes a computational runtime cost, reducing the throughput from ∼685–806 TPS in Standard ARF to ∼445–502 TPS in Dynamic CS-ARF. Both models fully utilize a single CPU core (∼99%). However, a critical advantage of Dynamic CS-ARF emerges in its RAM footprint. While Standard ARF expands extensively across the majority class space (growing up to 1.51 MB), Dynamic CS-ARF produces a significantly more compact ensemble (under 0.15 MB). Because the oversampling heavily re-evaluates the same rare minority instances, the trees update their internal node statistics without triggering excessive structural splits, creating a highly memory-efficient topology.

Despite the reduction in TPS, achieving ∼500 TPS on a single-threaded Python process indicates strong industrial feasibility. For context, major credit card networks like Visa process an average of 1,700 TPS globally. Because stream-based trees evaluate transactions independently, Dynamic CS-ARF can be straightforwardly parallelized across a modern 4- or 8-core CPU array to comfortably exceed global transaction velocities, making it highly viable for real-time industrial deployment.

4.6 Limitations

While the proposed Dynamic CS-ARF demonstrates significant improvements in imbalanced and drifting environments, several limitations must be acknowledged. Primarily, as demonstrated in our latency experiments, the algorithm's heavy reliance on real-time feedback estimators renders it vulnerable to extreme verification delays. Second, although Dynamic CS-ARF successfully auto-tunes its cost boundary and recovers rare anomalies, the reliance on dynamic Poisson bagging iterations naturally incurs a computational overhead, meaning deployment in microsecond-level frequency trading environments would require hardware-level parallelization.

5 Conclusion

This paper addressed the dual challenge of concept drift and high class imbalance in real-time financial fraud detection streams. We proposed the Dynamic Cost-Sensitive Adaptive Random Forest (Dynamic CS-ARF), which mitigates minority class exclusion via a dynamic *Implicit Online Oversampling* mechanism. Through prequential benchmarking against six state-of-the-art baselines across six industrial and benchmark datasets, we demonstrated that standard adaptive algorithms suffer performance degradation—frequently yielding very low Recall and F_2-Scores under high imbalance (e.g., 0.13% to 3.5%).

By continuously modulating an asymmetric cost penalty within the Poisson bagging distribution based on real-time stream statistics, Dynamic CS-ARF successfully recovers rare fraud instances that static and standard models frequently miss. Most importantly, Dynamic CS-ARF outperforms competitive undersampling models (e.g., UOB) in the critical F_2-Score metric because it avoids extensive majority-class deletion, preserving global Precision and preventing the high false alarm rates that hinder undersampled networks. Dynamic CS-ARF empirically achieved superior F_2-Scores across multiple evaluated streams, delivering the robust adaptive performance essential for protecting real-world financial networks.

While this dynamic oversampling delivers a decisive advantage in minority-class detection, it introduces a higher computational overhead compared to unmodified algorithms.

Furthermore, although formal statistical significance across our small dataset corpus requires expansion in future evaluation, the results demonstrate strong practical utility for industrial deployment without the need for manual hyperparameter tuning. Future work will explore hardware-parallelized variations of Dynamic CS-ARF for ultra-low latency trading environments.

References

[1] M. Abbasi, S. Lopez Florez, A. Shahraki, A. Taherkordi, J. Prieto, and J.M. Corchado. Class Imbalance in Network Traffic Classification: An Adaptive Weight Ensemble-of-Ensemble Learning Method. *IEEE Access*, 13:26171–26192, 2025.

[2] Gabriel Jonas Aguiar, Bartosz Krawczyk, and Alberto Cano. Cost-sensitive adaptive random forest for imbalanced data streams. *IEEE Transactions on Knowledge and Data Engineering*, 2023.

[3] K.I. Al-Daoud and I.A. Abu-AlSondos. Robust AI for Financial Fraud Detection in the GCC: A Hybrid Framework for Imbalance, Drift, and Adversarial Threats. *J. Theor. Appl. Electron. Commer. Res.*, 20(2), 2025.

[4] H.M.R. Al Lawati, A. Zainal, B.A.S. Al-Rimy, M. Al-Azawi, M.N. Kassim, S.A. Almalki, and T.A. Alghamdi. An Integrated Preprocessing and Drift Detection Approach With Adaptive Windowing for Fraud Detection in Payment Systems. *IEEE Access*, 13:92036–92056, 2025.

[5] R. Attivilli and A. Arul Jothi. Serverless Stream-Based Processing for Real Time Credit Card Fraud Detection Using Machine Learning. In Chakrabarti S. and Paul R., editors, *IEEE World AI IoT Congr., AIIoT*, pages 434–439. Institute of Electrical and Electronics Engineers Inc., 2023. Journal Abbreviation: IEEE World AI IoT Congr., AIIoT.

[6] B. Bayram, B. Koroglu, and M. Gonen. Improving Fraud Detection and Concept Drift Adaptation in Credit Card Transactions Using Incremental Gradient Boosting Trees. In Wani M.A., Luo F., Li X., Dou D., and Bonchi F., editors, *Proc. - IEEE Int. Conf. Mach. Learn. Appl., ICMLA*, pages 545–550. Institute of Electrical and Electronics Engineers Inc., 2020. Journal Abbreviation: Proc. - IEEE Int. Conf. Mach. Learn. Appl., ICMLA.

[7] A. Bernardo and E.D. Valle. SMOTE-OB: Combining SMOTE and Online Bagging for Continuous Rebalancing of Evolving Data Streams. In Chen Y., Ludwig H., Tu Y., Fayyad U., Zhu X., Hu X.T., Byna S., Liu X., Zhang J., Pan S., Papalexakis V., Wang J., Cuzzocrea A., and Ordonez C., editors, *Proc. - IEEE Int. Conf. Big Data, Big Data*, pages 5033–5042. Institute of Electrical and Electronics Engineers Inc., 2021. Journal Abbreviation: Proc. - IEEE Int. Conf. Big Data, Big Data.

[8] Albert Bifet and Ricard Gavaldà. Learning from time-changing data with adaptive windowing. In *Proceedings of the 2007 SIAM international conference on data mining*, pages 443–448. SIAM, 2007.

[9] Albert Bifet and Ricard Gavaldà. Adaptive learning from evolving data streams. In N.M. Adams, C. Robardet, A. Siebes, and J.-F. Boulicaut, editors, *Advances in Intelligent Data Analysis VIII*, volume 5772 of *Lecture Notes in Computer Science*. Springer, Berlin, Heidelberg, 2009.

[10] Rok Blagus and Lara Lusa. Smote for high-dimensional class-imbalanced data. *BMC bioinformatics*, 14:1–16, 2013.

[11] Dariusz Brzeziński and Jerzy Stefanowski. Online classifiers for imbalanced data streams. *Machine Learning*, 2021.

[12] C.-T. Chen, C. Lee, S.-H. Huang, and W.-C. Peng. Credit Card Fraud Detection via Intelligent Sampling and Self-supervised Learning. *ACM Trans. Intell. Syst. Technolog.*, 15(2):1–29, 2024.

[13] Y. Chen, X. Yang, and H.-L. Dai. Cost-sensitive continuous ensemble kernel learning for imbalanced data streams with concept drift. *Knowl Based Syst*, 284, 2024.

[14] Z. Cui, H. Tian, and H. Shen. Effective Density-Based Concept Drift Detection for Evolving Data Streams. In Park J.S., Takizawa H., Shen H., and Park J.J., editors, *Lect. Notes Electr. Eng.*, volume 1112 LNEE, pages 190–201. Springer Science and Business Media Deutschland GmbH, 2024. Journal Abbreviation: Lect. Notes Electr. Eng.

[15] Andrea Dal Pozzolo, Giacomo Boracchi, Olivier Caelen, Cesare Alippi, and Gianluca Bontempi. Credit card fraud detection: a realistic modeling and a novel learning strategy. *IEEE transactions on neural networks and learning systems*, 29(8):3784–3797, 2018.

[16] Janez Demšar. Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, 7(Jan):1–30, 2006.

[17] D. Dhiman, A. Bisht, A. Kumari, D.H. Anandaram, S. Saxena, and K. Joshi. Online Fraud Detection using Machine Learning. In *Int. Conf. Artif. Intell. Smart Commun., AISC*, pages 161–164. Institute of Electrical and Electronics Engineers Inc., 2023. Journal Abbreviation: Int. Conf. Artif. Intell. Smart Commun., AISC.

[18] Pedro Domingos and Geoff Hulten. Mining high-speed data streams. In *Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining*, pages 71–80, 2000.

[19] Aurora Esteban, Alberto Cano, Amelia Zafra, and Sebastián Ventura. Hoeffding adaptive trees for multi-label classification on data streams. *Knowledge-Based Systems*, 304:112561, 2024.

[20] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Comput. Surv.*, 46(4), March 2014.

[21] Heitor M Gomes, Albert Bifet, Jesse Read, Jean Paul Barddal, Fabrício Enembreck, Bernhard Pfahringer, Geoff Holmes, and Talel Abdessalem. Adaptive random forests for evolving data stream classification. *Machine Learning*, 106(9):1469–1495, 2017.

[22] Addison Howard, Bernadette Bouchon-Meunier, IEEE CIS, inversion, John Lei, Lynn@Vesta, Marcus2010, and Prof. Hussein Abbass. Ieee-cis fraud detection. `https://kaggle.com/competitions/ieee-fraud-detection`, 2019. Kaggle.

[23] H. Li. Credit Card Fraud Detection Based on Combination of Sparse Autoencoder and Support Vector Machine. In *ACM Int. Conf. Proc. Ser.*, pages 1252–1255. Association for Computing Machinery, 2022. Journal Abbreviation: ACM Int. Conf. Proc. Ser.

[24] Lucas Loezer, Fabriçio Enembreck, Jean Paul Barddal, and Alceu de Souza Britto. Cost-sensitive learning for imbalanced data streams. In *Proceedings of the 35th Annual ACM Symposium on Applied Computing*, 2020.

[25] Edgar Alonso Lopez-Rojas and Stefan Axelsson. Banksim: A bank payments simulator for fraud detection research. In *26th European Modeling and Simulation Symposium, EMSS 2014*, pages 144–152. Dime University of Genoa, 2014.

[26] Edgar Alonso Lopez-Rojas, A Elmir, and Stefan Axelsson. Paysim: A financial mobile money simulator for fraud detection. In *The 28th European Modeling and Simulation Symposium-EMSS, Larnaca, Cyprus*, 2016.

[27] F. Moradi, M. Tarif, and M. Homaei. Ensemble-Based Fraud Detection: A Robust Approach Evaluated on IEEE-CIS. In *Int. Conf. Comput. Knowl. Eng., ICCKE*. Institute of Electrical and Electronics Engineers Inc., 2025. Journal Abbreviation: Int. Conf. Comput. Knowl. Eng., ICCKE.

[28] Nasdaq Verafin. Global financial crime report 2026. Technical report, Nasdaq, 2026.

[29] T. Peng and Y.-J. Cheng. Hybrid Architecture Using Deep Auto-Encoder and Clustering Methods to Identify Diverse Transaction Anomalies. In Meen T.-H., editor, *Proc. IEEE Int. Conf. Comput., Big-Data Eng., ICCBE*, pages 584–589. Institute of Electrical and Electronics Engineers Inc., 2025. Journal Abbreviation: Proc. IEEE Int. Conf. Comput., Big-Data Eng., ICCBE.

[30] S. Priya and A. Uthra. Adaptive Synthetic Oversampling Algorithm for Handling Class Imbalance in Multi-Class Data Stream Classification. *J. Comput. Sci.*, 18(7):650–664, 2022.

[31] K. Sudarshana, C. MylaraReddy, and Z.A. Adhoni. Classification of Credit Card Frauds Using Autoencoded Features. In Rao B.N.K., Balasubramanian R., Wang S.-J., and Nayak R., editors, *Smart Innov. Syst. Technol.*, volume 315, pages 9–17. Springer Science and Business Media Deutschland GmbH, 2023. Journal Abbreviation: Smart Innov. Syst. Technol.

[32] V. Suggula and R.R. Goli. Real-Time Fraud Detection in Banking Transactions using Kafka Streaming and Machine Learning Ensemble. In *Int. Conf. Adv. Futur. Technol., ICAFT*. Institute of Electrical and Electronics Engineers Inc., 2025. Journal Abbreviation: Int. Conf. Adv. Futur. Technol., ICAFT.

[33] Y. Sun, M. Li, L. Li, H. Shao, and Y. Sun. Cost-Sensitive Classification for Evolving Data Streams with Concept Drift and Class Imbalance. *Comput. Intell. Neurosci.*, 2021, 2021.

[34] Y. Sun, Y. Sun, and H. Dai. Two-stage cost-sensitive learning for data streams with concept drift and class imbalance. *IEEE Access*, 8:191942–191955, 2020.

[35] P. Tomar, S. Shrivastava, and U. Thakar. Ensemble Learning based Credit Card Fraud Detection System. In *Conf. Inf. Commun. Technol., CICT*. Institute of Electrical and Electronics Engineers Inc., 2021. Journal Abbreviation: Conf. Inf. Commun. Technol., CICT.

[36] G. Venkatasubbu. Streaming Intelligence Real Time Fraud Detection in Retail Payments Using Machine Learning. In *Int. Conf. Eng. Emerg. Technol., ICEET*. Institute of Electrical and Electronics Engineers Inc., 2025. Journal Abbreviation: Int. Conf. Eng. Emerg. Technol., ICEET.